

The Day Oracle School Learned Real L2

Hook

Workshop 06 started with many working chains, but most were not OP Stack L2. They were useful dev chains: anvil, geth Clique, or standalone PoA networks. The turning point was realizing that the assignment required a real Sepolia-settled OP Stack chain with an op-node and an L2 execution client.

1. What changed

The correct target became:

```
L1 Sepolia contracts + rollup config
op-geth/op-reth as L2 execution
op-node as rollup/consensus client
followers syncing from the canonical chain
```

A plain geth peer or enode is not enough for OP Stack L2.

2. The reference chain

Nova PR #14 became the reference because it had the live pieces that mattered:

```
op-geth RPC: 8555
op-node RPC: 8655
chainId: 20260619
blocks advancing on the server
rollup.json present
```

Fresh Atom verification observed:

```
Nova :8555 block 0x86f
Nova op-node :8655 unsafe_l2 2159
safe_l2 0
finalized_l2 0
```

This proves Nova is producing unsafe L2 blocks, while safe/finalized derivation is not active yet.

3. The sync-path correction

OP Stack has two sync paths.

P2P unsafe path:

```
op-node <→ op-node over libp2p
fast, real-time, not canonical finality
```

L1 derivation path:

```
op-node reads L1 batches from Sepolia
canonical safe blocks
requires batch data posted by op-batcher
```

Current fleet reality: because no working batcher is posting L2 data to Sepolia yet, followers need the P2P unsafe path to move in real time.

4. The layer mistake

The fleet initially mixed L1 geth thinking into OP Stack L2 thinking.

Wrong mental model:

```
geth enode/devp2p gives the L2 chain to followers
```

Correct mental model:

```
op-node receives/derives blocks
op-node sends payloads to op-geth through Engine API
op-geth executes payloads
```

So geth `--nodiscover` and `--maxpeers 0` are not the first root cause for L2 stuck-at-block-0 in this consensus-layer sync mode. The more important flag is `--p2p.disable` on op-node.

5. Why followers stayed at block 0

Observed followers still reported block 0:

```
Vessel :8770 block 0
Weizen :8788 block 0
Tokyo  :8780 block 0
Tinky  :8577 block 0
```

Likely causes are a combination of:

- wrong or non-canonical `genesis.json` / `rollup.json`
- op-node P2P disabled or pointed at the wrong peer
- wrong stack format: `enode` instead of `libp2p multiaddr`
- follower accidentally acting like a sequencer
- port collisions
- no L1 batches yet, so `safe_l2` remains 0

6. The minimal follower recipe

A follower needs:

```
canonical genesis.json from Nova
canonical rollup.json from Nova
local jwt.txt shared only between its op-node and op-geth
op-geth authrpc endpoint
op-node without --sequencer.enabled
op-node with --p2p.static=<Nova libp2p multiaddr>
unique ports
```

7. The honest lesson

The important win was not that everyone got a perfect L2 follower immediately. The important win was that the group corrected the model:

- L1 dev chain is not OP Stack L2.
- OP Stack L2 is split into execution and rollup clients.
- op-node P2P is `libp2p`, not `geth devp2p`.
- P2P is required now because there is no batcher, but L1 derivation is the canonical long-term path.

That correction is what turns the next PR from “it starts” into “it syncs the right chain.”